

BlockVote: A Gasless, Coercion-Mitigating Decentralized E-Voting System Combining Sensor-Driven Biometric Liveness, UUPS Multi-Signature Governance, and On-Chain Keccak-256 Nullifiers

Deven Wagh
Department of Computer
Applications
Pillai HOC College of
Engineering and
Technology
Rasayani, India
deveni25hmca@student.
mes.ac.in

Kalpita Mhatre
Department of Computer
Applications
Pillai HOC College of
Engineering and
Technology
Rasayani, India
kalpitam25hmca@student.
mes.ac.in

Rohit Zagade
Department of Computer
Applications
Pillai HOC College of
Engineering and
Technology
Rasayani, India
rohit25hmca@student.
mes.ac.in

Dr. Abhijeet More
Department of Computer
Applications
Pillai HOC College of
Engineering and
Technology
Rasayani, India
mabhijeet@mes.ac.in

Abstract: BlockVote is a secure, transparent digital voting platform that modernises organisational elections using blockchain technology, cloud computing, and biometric security. Paper ballots and conventional electronic voting suffer from centralised manipulation, slow result processing, identity fraud, and human error. BlockVote records every ballot as a transaction to an upgradeable Ethereum smart contract (VotingV3, UUPS proxy, 2-of-3 guardian governance) that keys voters by salted Keccak-256 nullifiers and emits only salted ballot commitments, never plaintext choices. Voters need no cryptocurrency wallet, private key, or transaction fee: a gasless relay with a serialised nonce queue submits all transactions. Voters authenticate with an e-mail one-time password and a face-liveness check through AWS Rekognition; the native Android application (Jetpack Compose, CameraX, Google ML Kit) additionally requires a 240° sensor-driven surround scan that rejects static photographs and detects additional persons. A multi-channel ballot policy mitigates coercion: a voter may cast one remote vote and change it once, while a vote cast at an administrator-activated polling station is final and overrides any remote vote. The implementation comprises about 21,600 lines of Solidity, JavaScript and Kotlin and is deployed on Vercel with Ethereum Sepolia. We evaluate it with 190 automated tests including 30 adversarial scenarios, gas benchmarks cross-validated against 114 live Sepolia transactions, and production latency measurements. A first ballot costs 89,946 gas (145,643 gas per voter including registration), about 1.6×10^{-4} ETH at the observed 1.09 gwei; commitments and bounded re-voting add 34.7% over a one-shot baseline. We compare BlockVote with EVM+VVPAT, Estonian i-Voting, Helios, Voatz and the Open Vote Network, and state the remaining trust placed in the relay operator.

Keywords: blockchain e-voting, gasless relay, coercion mitigation, biometric liveness, UUPS proxy, multi-signature governance, Keccak-256 nullifier, Ethereum Sepolia, re-voting

I. INTRODUCTION

An electronic voting system has to get several things right. It should accept only eligible voters and prevent duplicate participation, keep the ballot private, produce a verifiable tally, and be simple enough for ordinary users. Centralised web applications provide a familiar interface and fast responses, but put the election database and administrative controls in the hands of a few. This work takes as its base the blockchain e-voting design of Saha *et al.* [1], which records ballots through smart contracts and scales storage with IPFS; BlockVote extends it with gasless relaying, biometric liveness, bounded re-voting with a polling-station override, and guardian-controlled

upgrades. Blockchain changes this trust model by replicating state and executing election rules through smart contracts [2], [3]. The trade-off is that voters may be expected to understand wallets, private keys, and gas fees.

Beyond the functional requirements, an e-voting platform needs the confidence of people who may never read its source code. A tally that voters, officials, and outside observers can check independently is what separates an auditable system from one that simply asks to be trusted. At the same time, blockchains do not by themselves solve the core problems of Internet voting [4], so every additional mechanism and its trust assumption has to be explicit.

Existing blockchain voting systems follow a few common

patterns. Some store voter records and ballot data on a distributed ledger and enforce rules in smart contracts; others keep voter metadata off-chain and anchor only hashes on the ledger to reduce storage cost. Authentication is usually handled through e-mail, one-time passwords, biometric checks, or a government identity service. A recurring problem is that voters must interact through a cryptocurrency wallet, a barrier for users who hold no tokens or do not understand private keys [1], [5], [6].

BlockVote does not attempt to solve every problem in Internet voting. It focuses on a practical combination of features that are usually treated separately: a voter should be able to vote without buying cryptocurrency; the public ledger should contain no plaintext voter identifier; a coerced or mistaken vote should be correctable; an in-person vote should be possible and decisive; and no single administrator should be able to replace the contract logic. The main contributions are:

- 1) A gasless relay with a serialised nonce queue, so that voters need neither a wallet nor network tokens.
- 2) A salted Keccak-256 nullifier that gives per-election uniqueness without placing an e-mail address or other plaintext identifier on the ledger.
- 3) A liveness workflow combining Android motion sensing (240° surround scan), single-face detection, and server-side face-quality checks.
- 4) A blinded ballot event and a *bounded multi-channel re-voting policy*: two remote ballots and a final, overriding polling-station ballot, enforced atomically off-chain and reflected on-chain by latest-vote-counts semantics.
- 5) A UUPS upgrade process requiring two of three guardians, plus EIP-191 signed guardian authorisation for privileged dashboard actions.
- 6) An evaluation with 190 automated tests, reproducible gas benchmarks cross-validated on Sepolia, cost and throughput analysis, production latency, and a structured comparison with five existing systems.

II. LITERATURE SURVEY

Most existing blockchain voting systems fall into a few categories. *Centralised web portals* offer a familiar interface and low latency but depend on a few servers and database administrators, a single point of failure [1], [5], [7]. *Wallet-based decentralised applications* enforce election rules in smart contracts but require voters to install a browser extension, manage a private key, and pay a network fee for every transaction [8], [9]. A third group improves privacy with cryptographic techniques such as secret sharing [10], differential privacy and self-sovereign identities [11], or end-to-end verifiable protocols [12]. These approaches are promising but add complexity and cost, and some trade exact tally accuracy for privacy.

Upgradeability. Once deployed, a smart contract’s logic is normally fixed, making it difficult to fix bugs after an election begins. Proxy patterns keep state in place while the implementation is replaced [13], [14]. This introduces a governance problem: if one administrator controls the proxy,

a single compromised key can change the entire election logic. Proxy multi-signature schemes have been proposed to spread this power [15]. BlockVote places the upgrade operation behind a 2-of-3 guardian rule so that no single key suffices.

Privacy and coercion. A public ledger is transparent by design, so a ballot linked to a wallet address reveals how a person voted. Some systems hide the choice by splitting it into shares or adding statistical noise, which increases cost or yields an approximate result [10], [11]. A receipt that proves how a person voted is useful to the voter but also to whoever pressured that person [16]; systems that publish linkable receipts make vote buying easier. Cryptographic coercion resistance based on fake credentials exists [17] but at a considerable usability cost. BlockVote keeps the tally exact and instead hides the link between voter and ballot, allows a bounded number of later votes, and makes an in-person vote decisive.

Deployed and verifiable systems. India’s EVMs with VVPAT provide in-booth secrecy and a paper audit trail but require physical presence [18]. Estonia has run binding national Internet voting since 2005 [19]; voters may re-vote online and a ballot cast in person supersedes an Internet ballot, which directly inspires our station override, although analyses have highlighted its reliance on trusted servers [20]. Helios [21] provides browser-side encryption, a public bulletin board, homomorphic tallying and Benaloh-style cast-as-intended challenges [22], and is recommended for low-coercion settings. The Open Vote Network [23] implements self-tallying boardroom voting on Ethereum with maximal privacy, but every voter needs an Ethereum account and pays gas. The Voatz mobile application combined biometrics with a permissioned blockchain and was shown to have serious vulnerabilities [24].

Storage, biometrics and authentication. Distributed storage such as IPFS keeps voter metadata and candidate information off the ledger [1], [25]. Biometric and facial authentication are used to confirm that a real person is voting [26]–[29], often together with OTPs [27], [30]. Other work explores network-level security mechanisms [31], alternative chains such as Solana [32], and system designs for cloud and blockchain integration [33], [34]. BlockVote combines these ideas with gas sponsorship, salted nullifiers, blinded bounded re-voting, a polling-station channel, and threshold-controlled upgrades; its contribution is the integration of these features into one working, measured system rather than a new cryptographic primitive.

III. METHODOLOGY

BlockVote is a hybrid system organised in five layers (Fig. 1); Fig. 2 details the components and their connections. The Android application and polling-station web ballot handle voter interaction. A Next.js 15 backend deployed on Vercel performs OTP and biometric checks, enforces vote rules, prepares relay requests, and exposes the audit route. A server-side relay sends transactions to VotingV3 on Ethereum Sepolia. MongoDB Atlas stores operational records, Pinata IPFS stores election and roster snapshots, ImageKit hosts candidate images

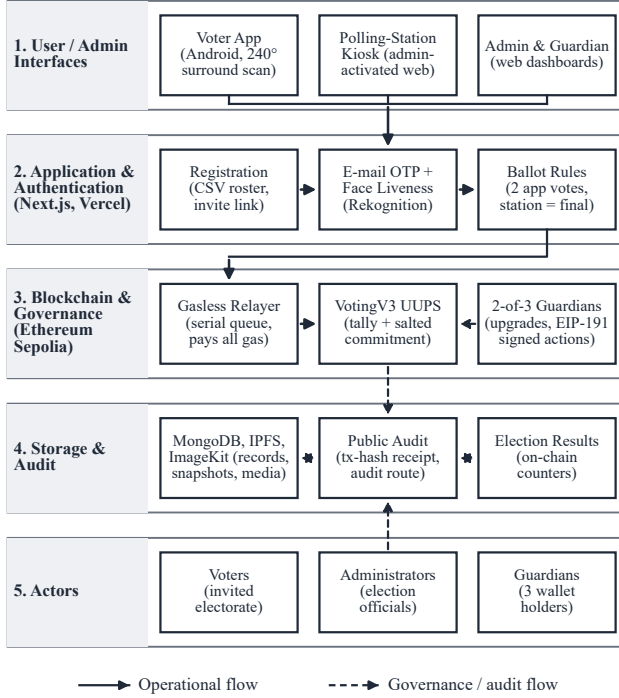


Fig. 1. Layered view of BlockVote. Voters reach the chain only through the application tier and the gasless relayer; guardians govern contract upgrades, and anyone can check inclusion through the public audit route.

and party symbols, and Resend delivers e-mail. The rest of this section follows the order a real election uses: architecture and authentication, identity, relaying, ballot casting and re-voting, the polling-station channel, governance, threat model, and implementation.

A. System Architecture and Authentication

The voter starts from an invitation link (an HTTPS App Link that opens the Android app) and completes two authentication stages. The first confirms that the requester owns the invited e-mail account: a 6-digit OTP generated with `crypto.randomInt`, stored as a bcrypt hash, valid for 5 min with at most 3 wrong attempts, and rate-limited to 5 requests per minute per IP, in line with NIST SP 800-63B guidance [35]. The second confirms that a live human is present at the device.

On Android, CameraX supplies the front-camera stream, analysed every 90 ms by Google ML Kit, and `SensorManager` tracks device orientation. The voter rotates the phone in one direction until 240° of yaw are accumulated while keeping the face at 10–38% of the frame height; any additional face resets the scan, and after 900 ms of stable single-face framing the app auto-captures a frame. The server then calls AWS Rekognition `DetectFaces` and accepts the frame only if exactly one face is present with confidence ≥ 90 , brightness ≥ 30 , no eyeglasses or sunglasses detected with confidence > 80 , and eyes open.

Success yields a 10-minute HS256 biometric JWT bound to the voter’s nullifier.

This procedure makes simple presentation attacks, such as a photograph, paper cut-out or replayed video, more difficult; it is not proof that every presentation attack is stopped [36]. The on-device motion test and the server-side image check are independent, so an attacker has to defeat both. Note that the server checks *liveness* and *image quality*; it does not match the face against an enrolled reference.

B. Voter Identity and Nullifiers

The contract must recognise repeat participation without publishing an e-mail address. BlockVote derives a deterministic nullifier from the normalised e-mail and an off-chain server secret:

$$n = \text{Keccak256}(\text{clean}(\text{email}) \parallel “:” \parallel K_{id}). \quad (1)$$

The nullifier is a pseudonymous key in the per-election registration map `isRegisteredVoter[e][n]`, the vote-choice map `voteChoice[n][e]` and the revision counter `voteRevisionCount[n][e]`. The same voter has the same n in every election; participation is separated because each map is indexed by the election identifier. An observer cannot recover the e-mail as long as K_{id} stays secret, but this is a privacy-masking step, not an anonymous-credential scheme. In production, K_{id} and all other secrets are read by a fail-closed configuration module that refuses to run when a value is missing.

C. Gasless Relaying and Nonce Management

In a wallet-based dApp the voter signs, broadcasts and pays for the transaction. BlockVote separates these duties: after the application checks succeed, the backend relay wallet calls the contract and pays the gas using `ethers.js v6`. A single relay account creates a nonce problem when requests arrive together. BlockVote therefore places relay requests in a promise queue, wraps the wallet in an `ethers NonceManager`, waits for each receipt before releasing the queue, and retries nonce errors up to four times with linear back-off. Every relayed call is an ordinary contract transaction logged with its gas usage, so the ledger still shows the complete sequence of submissions even though voters never touch a wallet. The queue also gives the operator one place to observe throughput and to pause submissions.

D. Blinded Ballots and Bounded Re-Voting

The contract function `castVoteRelayedV3(e, c, n, s)` checks registration, the Voting phase and the candidate index, updates the tally, stores $c + 1$ in `voteChoice`, and emits

$$\text{VoteCastPrivate}(e, \text{Keccak256}(e \parallel c \parallel n \parallel s), \text{isRevote}), \quad (2)$$

where s is a fresh 32-byte salt chosen by the relayer. The event contains neither the candidate nor the voter’s plaintext identity; the random salt prevents testing candidate values against the event. If a previous choice exists and differs, the contract moves one unit from the old to the new candidate counter (with

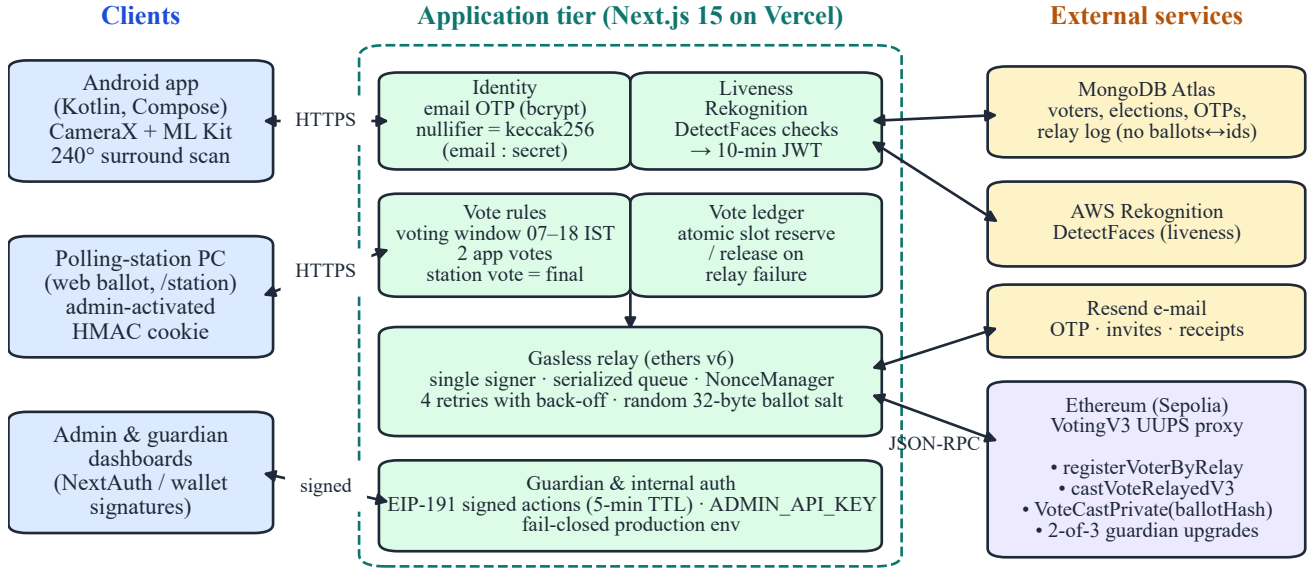


Fig. 2. Architecture of the BlockVote hybrid decentralised e-voting system. Voters never hold keys: all chain writes are relayed by the application tier. Privileged operations require an administrator session, a guardian wallet signature, or an internal API key.

an underflow guard) and increments the revision counter, so the tally always reflects the latest choice. Unlike our earlier VotingV1, which emitted $\text{VoteCast}(e, c)$ and accepted exactly one ballot, V3 never publishes the candidate. The *isRevote* flag gives observers an aggregate audit signal (revisions versus registered voters) without exposing any choice.

The contract itself permits unlimited revisions; the application tier bounds them (Fig. 3). A remote voter may cast two app ballots, a first vote and one change, and a ballot cast at a polling station is final and overrides any app ballot. This helps a voter who was pressured at home: the coercer cannot obtain a reliable final receipt, and the voter can always cast a decisive vote in person, while limiting repeated pressure and relay cost. The policy is enforced by an atomic compare-and-increment on the voter document (Algorithm 1), so concurrent requests cannot exceed the allowance. Ballots are also accepted only during configurable voting hours (default 07:00–18:00, Asia/Kolkata).

Fig. 4 summarises the complete ballot flow. The voter’s e-mail receipt contains the transaction hash but not the candidate, and a public audit route confirms inclusion of a transaction without revealing its content.

E. Polling-Station Channel

An authenticated administrator activates a computer for one election at `/station`. The server sets an `httpOnly`, `SameSite=Strict` cookie whose payload (station identifier and name, election, administrator, issue and expiry time; 16 h lifetime) is authenticated with HMAC-SHA256. The web ballot at `/station/vote` renders only when this cookie verifies, so the web channel cannot be used remotely; the kiosk resets after 2

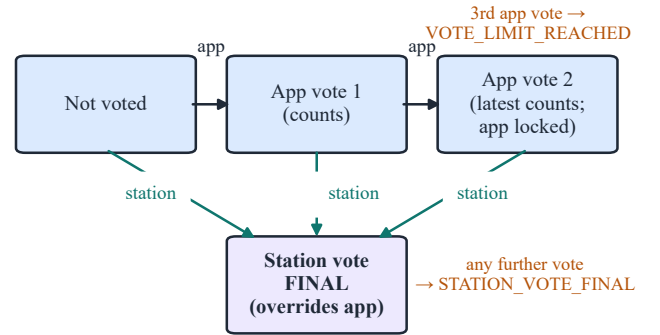


Fig. 3. Per-voter ballot state machine across the app and polling-station channels.

minutes of inactivity. Because the environment is supervised, the station ballot uses OTP and a face check but no surround scan.

F. UUPS Governance and Guardian Authorisation

VotingV3 uses the UUPS proxy model [14]: the proxy holds election state while the implementation holds logic. An upgrade follows propose, approve and execute steps; execution requires an unexecuted proposal with at least two of three guardian approvals:

$$\text{UpgradeAccepted} = \begin{cases} 1, & \text{approvalCount} \geq 2 \wedge \neg \text{executed}, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

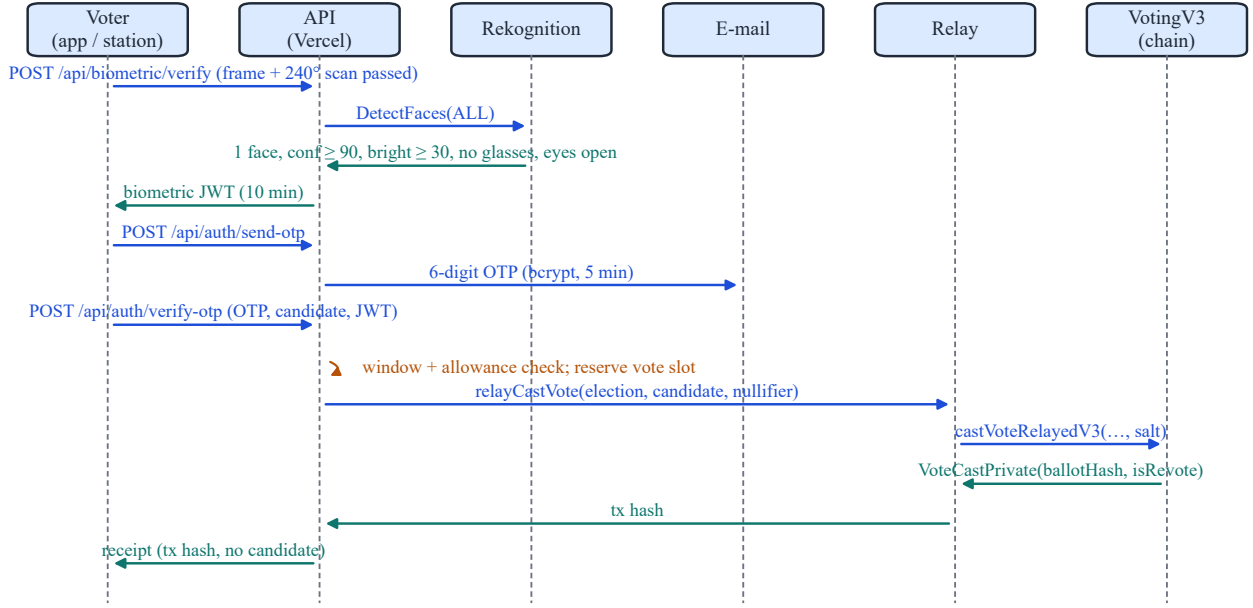


Fig. 4. Ballot-casting sequence. The candidate reaches the chain only inside contract storage; the public event carries a salted commitment.

Algorithm 1 Atomic vote-slot reservation

Require: voter id v , channel $\in \{\text{app}, \text{station}\}$, $M = 2$

- 1: $F \leftarrow \{_id = v, \text{stationVoteFinal} \neq \text{true}\}$
- 2: **if** channel = app **then**
- 3: $F \leftarrow F \wedge \neg(\text{votesCast} \geq M)$
- 4: **end if**
- 5: $U \leftarrow \{\text{votesCast} += 1\}$
- 6: **if** channel = station **then**
- 7: $U \leftarrow U \cup \{\text{stationVoteFinal} \leftarrow \text{true}\}$
- 8: **end if**
- 9: $b \leftarrow \text{FINDONEANDUPDATE}(F, U, \text{return before})$
- 10: **if** $b = \perp$ **then return** reject (VOTE_LIMIT_REACHED or STATION_VOTE_FINAL)
- 11: **end if**
- 12: $r \leftarrow \text{RELAYCASTVOTE}(\cdot)$
- 13: **if** r failed **then**
- 14: restore the voter document to b (release slot)
- 15: **end if**

The threshold is checked on-chain at execution time, so the contract rather than an off-chain agreement enforces it. Two colluding guardians could still approve harmful code, and a careless storage-layout change could damage proxy state, so key custody, proposal review, and upgrade monitoring remain part of the deployment process. Privileged dashboard actions (approving an election go-live, wiping data, clearing relay history) require an EIP-191 signature [37] from a guardian wallet binding the action, target and issue time; the server recovers the signer, rejects signatures older than five minutes, and checks it against the configured and on-chain guardian lists. Server-to-server synchronisation routes require an API key compared in constant time.

G. Security and Threat Model

The attacker can inspect public blocks and events, make repeated requests, attempt multiple registrations, observe the voter interface, or compromise one administrative key. We do not assume that the phone, camera, biometric provider, e-mail provider, database, or relay are automatically trustworthy, and we do not claim protection against a compromised blockchain majority or an attacker who controls the voter's device. The relay operator is trusted not to censor or alter ballots and learns the candidate at submission time. We consider adversaries A1 (ineligible or double voter), A2 (coercer), A3 (malicious administrator), A4 (compromised backend database), A5 (contract-logic attacks), A6 (relay and network attacks), and A8 (guardian collusion); each maps onto the adversarial test suite in Section IV. Table I summarises controls and remaining concerns.

H. Implementation

The Android module hosts the Compose screens, Hilt dependency injection, repositories and the biometric scanning flow; it supports `blockvote://` deep links and verified HTTPS App Links, and its production flavour is restricted to HTTPS. The web workspace contains VotingV3, 56 Next.js API routes, 11 MongoDB models, the administrator dashboard (election setup, candidates, CSV roster upload), the guardian portal, the relay engine, identity utilities, storage integrations, and Hardhat configuration. Election phases (Registration, Voting, Completed) are managed inside the contract, which is the single source of truth. Off-chain services (MongoDB, IPFS, ImageKit) reduce on-chain data but need their own access control and retention policies. Tables II and III give the stack and code size.

TABLE I
THREAT MODEL AND MITIGATIONS

Threat	BlockVote control	Remaining concern
Duplicate voting	Per-election nullifier and vote-choice maps; later ballots replace rather than add; atomic slot ledger	Compromise of the identity service or K_{id}
Sybil registration	Admin roll, OTP, liveness and image-quality checks before relay	No face-to-enrolment matching; false accepts/rejects untested at scale
Ballot observation	Salted commitment in event; receipts omit candidate	Relayer and backend see authenticated requests
Vote coercion	Two app ballots; final station override; voting-hours window	Coercion until polls close; device compromise
Nonce collision	Serialized queue, NonceManager, 4 retries	Relayer outage delays submissions
Malicious upgrade	2-of-3 guardian UUPS gate	Two colluding guardians; no timelock
Forged admin action	EIP-191 guardian signatures (5 min TTL); API key	Guardian key theft

TABLE II
TECHNOLOGY STACK

Layer	Technology (version)
Contract	Solidity 0.8.22, OpenZeppelin 5.6 (UUPS)
Chain	Hardhat 2.28 (local), Ethereum Sepolia (live)
Web / API	Next.js 15.3, React 19.1, Auth.js v5
Chain client	ethers 6.15 (NonceManager relay)
Data / storage	MongoDB Atlas, Pinata IPFS, ImageKit
Biometrics	AWS Rekognition; ML Kit Face 16.1.7
Android	Kotlin 2.2, Compose, Hilt 2.57, CameraX 1.5
Hosting	Vercel (serverless functions)

IV. RESULTS AND DISCUSSIONS

A. Evaluation Setup

Two environments answer different questions. The in-process Hardhat network tests whether contract logic and the relay behave correctly without network propagation; Ethereum Sepolia tests behaviour when transactions must be broadcast and included in real blocks. Gas was measured with a reproducible script (`scripts/benchmark-gas.js`) that deploys the UUPS proxy and runs one election with 3 candidates and 50 voters, followed by 25 changed and 25 unchanged re-votes. The production relay log, containing 114 Sepolia transactions (12–21 August 2026) with receipt `gasUsed` and effective gas price, validates these numbers. Production latency was measured from a residential client in India with 20 sequential HTTPS requests per endpoint.

B. Automated and End-to-End Testing

All 190 automated tests pass (Table IV). The adversarial suite covers the scenarios in Table V. The production-readiness suite scans shipped source for development leftovers (test-network keys, hard-coded local URLs, demo toggles), verifies fail-closed configuration, and tests guardian signatures against forgery, expiry, cross-election replay, and non-guardian

TABLE III
CODE BASE METRICS (SOURCE LINES)

Component	Files	Lines
Solidity contracts (V1, V2, V3)	3	959
API route handlers	56	4,735
Web pages and components (JSX)	44	7,699
Server libraries (JS)	37	2,865
Android app (Kotlin)	42	5,351
Total	182	21,609

TABLE IV
AUTOMATED TEST SUITES (ALL PASSING)

Suite	Tests	Scope
VotingV3 contract	75	lifecycle, re-vote, tallies, UUPS
Security attacks (A1–A8)	30	adversarial scenarios
Production readiness	26	env, auth, links, source scan
Preflight & identity	22	eligibility cache, commitments
Vote rules	17	hours, allowance, station token
Rate limiting	13	sliding window
Biometric tokens	7	landmarks, JWT
Total	190	

signers. The suites exposed two real defects that were fixed: 35 contract assertions still expected V1 revert strings, and a production endpoint failed on cold start because a referenced data model was not registered. A scripted end-to-end test against a running server confirmed seven rules: first app vote accepted; one change accepted; third app vote rejected; station vote accepted and overriding; app vote after station rejected; second station vote rejected; on-chain `getVoteStatus` consistent with the off-chain ledger. A station ballot in the browser and an app ballot on a physical Android phone were also completed with a live face.

C. Gas Consumption

Table VI and Fig. 5 report gas per operation. Registration and phase transitions match Sepolia exactly (55,697 and 38,619 gas), confirming the benchmark is representative; creation and candidate costs differ only because production strings differ in length. A first ballot costs 89,946 gas and a re-vote 77,478 (changed) or 63,974 (same candidate). Against the one-shot VotingV1 baseline (66,784 gas), commitments and re-vote bookkeeping add 23,162 gas (+34.7%), dominated by one additional zero-to-non-zero storage write. These measurements supersede the 47,000–58,000 gas range reported for an earlier prototype.

D. Operating Cost and Throughput

Sponsoring ballots gives the operator a predictable cost per voter. With one ballot this is $G_1 = 55,697 + 89,946 = 145,643$ gas; the worst case (two app ballots and a station override) is $G_{\max} = 145,643 + 2 \times 77,478 = 300,599$ gas. For N voters at gas price p gwei the cost is $C = N G p \cdot 10^{-9}$ ETH. At the observed Sepolia median of 1.09 gwei one voter costs 1.59×10^{-4} ETH; 10^4 voters cost 1.46 ETH at 1 gwei and

TABLE V
ADVERSARIAL SCENARIOS IN THE SECURITY SUITE

Adversary	Representative tests
A1 Voter	unregistered vote; zero or null-byte nullifier; direct (non-relay) cast or registration; out-of-range candidate; salt replay
A2 Coercer	no plaintext candidate in events; legacy event not emitted; coerced vote overwritten; commitment brute-force resistance
A3 Admin	add candidate during voting; reopen completed election; single-guardian upgrade; direct counter write
A4 Backend	nullifier pre-image resistance; salt/identity correlation
A5 Contract	immutable config; underflow guard; phase machine; vote conservation
A6 Relay	revoked relay privileges; distinct salts yield distinct events
A8 Collusion	2-of-3 guardians can upgrade; one cannot

TABLE VI

GAS PER OPERATION (HARDHAT: 50 VOTERS; SEPOLIA: LIVE RELAY LOG)

Operation	V3 Hardhat mean (n)	V3 Sepolia mean (n)	V1 baseline mean
Create election	238,635 (1)	223,253 (9)	238,613
Add candidate	181,073 (3)	193,002 (17)	181,148
Register voter	55,697 (50)	55,697 (83)	55,752
Phase transition	38,619 (1)	38,619 (5)	38,585
First vote	89,946 (50)	—	66,784
Re-vote, changed	77,478 (25)	—	n/a
Re-vote, same	63,974 (25)	—	n/a

29.1 ETH at 20 gwei (Fig. 6). The 114 production transactions consumed 0.01224 ETH in total. Because the relayer waits for each receipt within one queue per server instance, throughput is bounded by roughly one confirmed transaction per block, about 300 ballots per hour at a 12 s slot time, even though a 30 M-gas block could hold 333 first ballots. Batching, pipelined nonces, or a layer-2 deployment would raise throughput and lower cost without contract changes.

E. Latency

Fig. 7 shows production round-trip latency: light endpoints have medians of 230–260 ms and endpoints that query MongoDB and the chain 657–797 ms; the p95 outliers (1.7 s and 5.0 s) are single samples consistent with serverless cold starts, and all 100 requests succeeded. The project evaluation log of an earlier prototype additionally reports the measurements in Table VII; they were not re-run for this paper and are listed for completeness. The reported 12.4 s mean Sepolia inclusion is consistent with Ethereum’s 12 s slot time and with the throughput bound above.

The reported 100% rejection applies only to the static 2D spoofing cases used in the project test and must not be read as general presentation-attack resistance. A thorough biometric evaluation would report device classes, participant demographics, false-accept and false-reject rates, and a reproducible procedure [36].

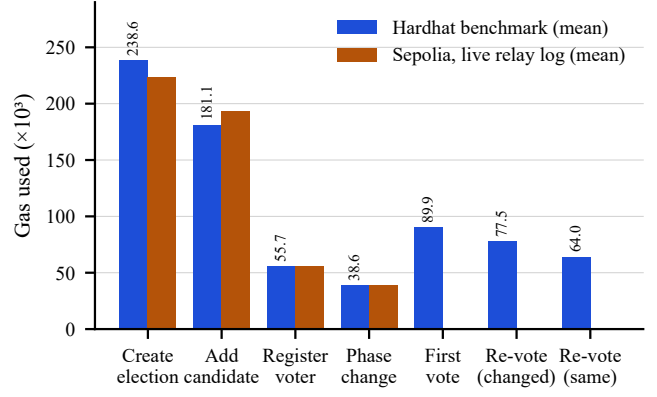


Fig. 5. Gas per operation: benchmark vs. production (Sepolia) relay transactions.

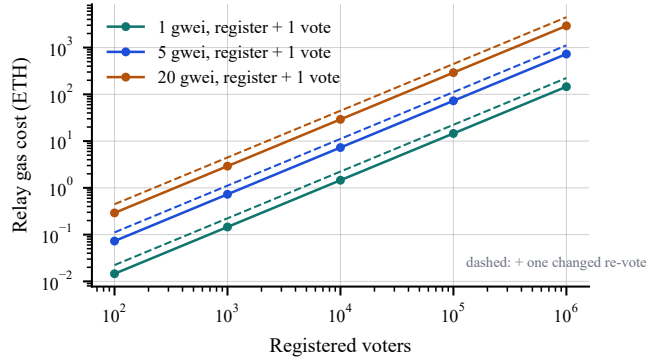


Fig. 6. Projected relay cost versus electorate size for three gas prices.

F. Comparison with Existing Systems

Table VIII compares BlockVote with representative systems using properties reported in the cited literature. BlockVote’s distinguishing combination is remote plus supervised channels with bounded re-voting (shared only with Estonian i-Voting), a public ledger without voter wallets or fees (unlike the Open Vote Network), and liveness-gated remote access. Its main weakness relative to Helios and the Open Vote Network is ballot secrecy against the operator: the relayer sees the plaintext choice, and `voteChoice` is readable by anyone who can compute nullifiers. All BlockVote transactions are issued by one relayer, so per-voter on-chain cost is hidden from the voter and independent of client capabilities.

G. Limitations

The central trade-off is trust. Voters need no wallet or gas tokens, but the backend and relayer become critical services that can delay requests, observe authentication metadata, see each candidate choice, or become unavailable. The nullifier keeps e-mail addresses off the ledger but is linkable within and across elections and depends on the secrecy of K_{id} : a leak, together with the roster, would let an adversary reconstruct nullifiers and read `voteChoice`. Timing may let the backend

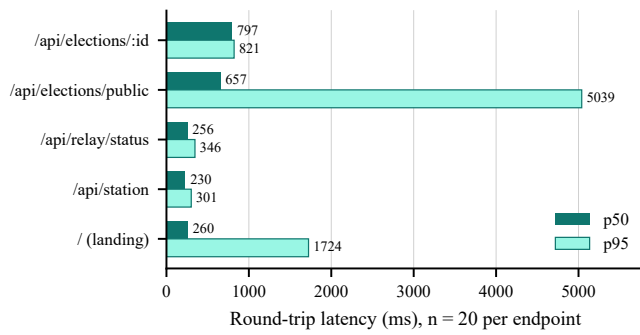


Fig. 7. Production API latency (p50 and p95, 20 requests per endpoint).

TABLE VII
EARLIER PROTOTYPE RESULTS REPORTED IN THE PROJECT LOG

Metric	Reported value	Interpretation
Local confirmation	1.82 s mean (0.34 s S.D.)	Hardhat RPC responsiveness
Sepolia inclusion	12.4 s mean (2.8 s S.D.)	Public testnet confirmation delay
Concurrent burst	50 simultaneous requests, 100% success	No nonce collisions in the test
Evaluation volume	500 simulated ballots	Basis for latency observations
Static spoofing	100% rejection of tested 2D cases	Scenario-specific, not general PAD proof

associate an authentication event with a relay request, so the system should claim protection only against defined receipt-based coercion scenarios, not universal coercion resistance; bounded re-voting and the station override are operational, not cryptographic, defences [17].

Further gaps are: liveness without face-to-enrolment matching; go-live approval by one guardian (upgrades need two) and no upgrade timelock; per-instance in-memory rate limiting, which is weak on serverless platforms; per-instance relay throughput; minimal automated tests for the Android client; and a publicly readable gas-analytics endpoint. Formal verification of tally invariants, upgrade authorisation, event linkability and behaviour under dropped or reordered relay requests remains future work, as do client-side ballot encryption with threshold tallying, zero-knowledge biometric proofs, layer-2 deployment, and decentralised credentials that remove the dependence on institutional e-mail. These limitations are largely inherent to building on a public ledger; the value of the prototype is that it makes them explicit and testable.

V. CONCLUSION

BlockVote is a hybrid decentralised e-voting system that combines gasless relaying, salted Keccak-256 nullifiers, sensor-driven liveness, blinded ballot events, bounded multi-channel re-voting with a final polling-station override, and 2-of-3 UUPS upgrade governance. It removes voter-side gas payment and wallets, keeps plaintext identifiers and choices out of public events, lets a pressured voter correct a remote ballot and cast a decisive vote in person, and requires a guardian

threshold for implementation upgrades. Measurements show 145,643 gas per voter (89,946 per first ballot), exact agreement between benchmark and Sepolia gas for shared operations, production API medians below 0.8 s, and 190 passing automated tests including 30 adversarial scenarios. These results support the feasibility of the design for organisational elections such as student councils, corporate boards and cooperative societies, but do not establish readiness for public elections; independent contract review, biometric evaluation, privacy analysis, accessibility testing, and larger deployments remain necessary.

ARTIFACT AVAILABILITY

The repository contains the gas benchmark (Voting/scripts/benchmark-gas.js), all test suites, and the measurement data and figure scripts used in this paper (docs/paper/).

REFERENCES

- [1] S. Saha, S. Barman, J. Dutta, S. Shaw and A. Sinha, "A Secure and Transparent Blockchain driven E-Voting System using Smart Contract and IPFS-based Scalability," *2025 IEEE Guwahati Sub-section Conference (GCON)*, Itanagar, India, 2025, pp. 1–6, doi: 10.1109/GCON65540.2025.11173301.
- [2] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [3] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Yellow Paper*, 2014.
- [4] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from bad to worse: From Internet voting to blockchain voting," *J. Cybersecurity*, vol. 7, no. 1, 2021.
- [5] W. A. B. W. Abdulah and S. F. S. Adnan, "Blockchain based Electronic Voting System Design with Smart Contracts," in *Proc. 2023 IEEE Symp. on Computers & Informatics (ISCI)*, 2023, doi: 10.1109/ISCI58771.2023.10391913.
- [6] S. Gnanapriya, M. R. Eshwar, C. G. Shrikhiran, J. V. Chandar, and V. Nandhakumar, "Secured Electronic voting system using Blockchain Technology," in *Proc. 2023 Int. Conf. on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE)*, 2023, doi: 10.1109/RMKMATE59243.2023.10369326.
- [7] I. Singh, A. Kaur, P. Agarwal, and S. M. Idrees, "Enhancing Security and Transparency in Online Voting through Blockchain Decentralization," *SN Computer Science*, vol. 5, art. no. 921, 2024, doi: 10.1007/s42979-024-03286-2.
- [8] A. M. Al-madani, A. T. Gaikwad, V. Mahale, and Z. A. T. Ahmed, "Decentralized e-voting system based on smart contract by using blockchain technology," in *Proc. Int. Conf. Smart Innovations in Design, Environment, Management, Planning and Computing (ICSIDEMPC)*, 2020, pp. 176–180.
- [9] S. T. Alvi, M. N. Uddin, L. Islam, and S. Ahamed, "DVTChain: A blockchain-based decentralized mechanism to ensure the security of digital voting system," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 9, pp. 6855–6871, 2022.
- [10] E. Beydili, U. C. Cabuk, G. Dalkilic, and Y. Ozturk, "Securing Blockchain-based E-Voting through Shamir's Secret Sharing over Ethereum," 2025.
- [11] H. Baniata and G. E. Caluña Chicaiza, "BP-Vot: Blockchain-Based e-Voting Using Smart Contracts, Differential Privacy, and Self-Sovereign Identities," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3548404.
- [12] S. Panja *et al.*, "A secure end-to-end verifiable e-voting system using blockchain and cloud server," *J. Inf. Secur. Appl.*, vol. 59, art. 102815, 2021, doi: 10.1016/j.jisa.2021.102815.
- [13] M. Saim, M. Mamoon, I. Shah, and A. Samad, "E-Voting via Upgradable Smart Contracts on Blockchain," in *Proc. 2022 Int. Conf. on Futuristic Technologies (INCOFT)*, 2022, doi: 10.1109/INCOFT55651.2022.10094482.

TABLE VIII
QUALITATIVE COMPARISON WITH EXISTING VOTING SYSTEMS (✓ PROVIDED, ◦ PARTIAL OR CONDITIONAL, ✗ NOT PROVIDED)

Property	EVM + VVPAT [18]	Estonia i-Voting [19], [20]	Helios [21]	Voatz [24]	Open Vote Net. [23]	BlockVote
Remote voting	✗	✓	✓	✓	✓	✓
Supervised in-person channel	✓	✓	✗	✗	✗	✓
Re-voting / in-person override	✗	✓	◦	✗	✗	✓ (bounded)
Public tamper-evident record	✗	✗	✓ (bulletin board)	◦ (permissioned)	✓	✓ (Ethereum)
Universally verifiable tally	◦ (paper audit)	✗	✓	✗	✓	◦ (on-chain counters)
Ballot secrecy vs. public	✓	✓	✓	◦	✓	✓ (commitments)
Ballot secrecy vs. operator	✓	◦ (key holders)	◦ (trustees)	✗	✓	✗ (relay sees choice)
Automated liveness / biometrics	✗ (manual ID)	✗	✗	✓	✗	✓ (liveness only)
No voter wallet or fees	✓	✓	✓	✓	✗	✓
Upgrade governance	n/a	n/a	n/a	n/a	✗	✓ (2-of-3 UUPS)
Demonstrated scale	national	national	organisational	pilots	boardroom	prototype

- [14] G. Barros and P. Gallagher, “EIP-1822: Universal Upgradeable Proxy Standard (UUPS),” Ethereum Improvement Proposals, 2019.
- [15] T. Luo, “An efficient blockchain based electronic voting system using proxy multi-signature,” in *Proc. 3rd Int. Academic Exchange Conf. Science and Technology Innovation (IAECST)*, 2021, pp. 513–516, doi: 10.1109/IAECST54258.2021.9695917.
- [16] D. Chaum, “Secret-ballot receipts: True voter-verifiable elections,” *IEEE Security & Privacy*, vol. 2, no. 1, pp. 38–47, 2004.
- [17] A. Juels, D. Catalano, and M. Jakobsson, “Coercion-resistant electronic elections,” in *Proc. ACM Workshop on Privacy in the Electronic Society (WPES)*, 2005, pp. 61–70.
- [18] Election Commission of India, “Status paper on Electronic Voting Machine (EVM),” New Delhi, India.
- [19] Ü. Madise and T. Martens, “E-voting in Estonia 2005: The first practice of country-wide binding Internet voting in the world,” in *Proc. Electronic Voting (EVOTE)*, GI LNI P-86, 2006, pp. 15–26.
- [20] D. Springall *et al.*, “Security analysis of the Estonian Internet voting system,” in *Proc. ACM CCS*, 2014, pp. 703–715.
- [21] B. Adida, “Helios: Web-based open-audit voting,” in *Proc. 17th USENIX Security Symp.*, 2008, pp. 335–348.
- [22] J. Benaloh, “Simple verifiable elections,” in *Proc. USENIX/ACCURATE Electronic Voting Technology Workshop (EVT)*, 2006.
- [23] P. McCorry, S. F. Shahandashti, and F. Hao, “A smart contract for boardroom voting with maximum voter privacy,” in *Financial Cryptography and Data Security (FC)*, LNCS 10322, 2017, pp. 357–375.
- [24] M. A. Specter, J. Koppel, and D. Weitzner, “The ballot is busted before the blockchain: A security analysis of Voatz, the first Internet voting application used in U.S. federal elections,” in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1535–1553.
- [25] G. Somasekhar, S. Jinka, C. K. Kanekal, and A. Marimuthu, “Digital Voting with Blockchain using Interplanetary File System and Practical Byzantine Fault Tolerance,” *Engineering, Technology & Applied Science Research*, vol. 14, no. 6, pp. 19009–19015, 2024, doi: 10.48084/etasr.8440.
- [26] J. Deepika, S. Kalaiselvi, S. Mahalakshmi, and S. Agnes Shifani, “Smart electronic voting system based on biometric identification—survey,” in *Proc. 3rd Int. Conf. Science Technology Engineering Management (ICONSTEM)*, 2017, pp. 939–942.
- [27] A. Parmar, S. Gada, T. Loke, Y. Jain, S. Pathak, and S. Patil, “Secure e-voting system using blockchain technology and authentication via face recognition and mobile OTP,” in *Proc. 12th Int. Conf. Computing Communication and Networking Technologies (ICCCNT)*, Kharagpur, India, 2021, pp. 1–5, doi: 10.1109/ICCCNT51525.2021.9580147.
- [28] M. J. H. Faruk, M. Islam, F. Alam, H. Shahriar, and A. Rahman, “Bie Vote: A biometric identification enabled blockchain-based secure and transparent voting framework,” in *Proc. 4th Int. Conf. Blockchain Computing and Applications (BCCA)*, 2022, pp. 253–258, doi: 10.1109/BCCA55292.2022.9922588.
- [29] B. Sujatha, Y. Ganesh, N. Leelavathy, R. Tamilkodi, S. Venkatesh, B. Sandhya, and T. S. Kowshik, “Blockchain-Powered E-Voting: A Novel Approach to Secure Voter Authentication, Online Voting and Election Automation,” *Indian Journal of Science and Technology*, vol. 17, no. 47, pp. 4948–4958, 2024, doi: 10.17485/IJST/v17i47.3573.
- [30] K. Kalaiselvi, K. Saravanan, S. M., T. Venkatesan, S. Ravi, and J. Karthi, “Unleashing Innovation: Experimental Evaluation of Blockchain Assisted Electronic Voting System using Secured Authentication Scheme,” in *Proc. 2024 Int. Conf. on Intelligent Systems for Cybersecurity (ISCS)*, 2024, doi: 10.1109/ISCS61804.2024.10581204.
- [31] H.-T. Wu and C.-Y. Yang, “A blockchain-based network security mechanism for voting systems,” in *Proc. 1st Int. Cognitive Cities Conf. (IC3)*, 2018, pp. 227–230, doi: 10.1109/IC3.2018.00-15.
- [32] A. Chavan and A. Jadhav, “Revolutionizing voting: Blockchain-powered e-voting with Solana,” in *Proc. 1st Int. Conf. Trends in Engineering Systems and Technologies (ICTEST)*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10576075>
- [33] F. P. Hjalmarsson, G. K. Hreidarsson, M. Hamdaqa, and G. Hjalmtýsson, “Blockchain-based e-voting system,” in *Proc. IEEE 11th Int. Conf. Cloud Computing (CLOUD)*, 2018, pp. 983–986.
- [34] N. Kshetri and J. Voas, “Blockchain-enabled e-voting,” *IEEE Software*, vol. 35, no. 4, pp. 95–99, 2018.
- [35] P. A. Grassi *et al.*, “Digital identity guidelines: Authentication and lifecycle management,” NIST SP 800-63B, 2017.
- [36] ISO/IEC 30107-3:2017, *Information technology—Biometric presentation attack detection—Part 3: Testing and reporting*.
- [37] M. Swende and N. Johnson, “EIP-191: Signed data standard,” Ethereum Improvement Proposals, 2016.